

For making the snake game using the console application we first know the game logic

1. In this game this snake is inside a grid that we created
 2. the snake changes the food to eat
 3. When it eats the food then the snake size increases and also score increases
 4. If the snake hit the wall or its own body then the game restart and the size of the snake is same as the starting of the game and also the score reset
 5. the food is randomly change in every instances
-

-Open visual studio the create a new project name snake

-create a new class name coordinate

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Snake
{
    internal class coordinate
    {
        private int x;
        private int y;

        public int X { get { return x; } }
        public int Y { get { return y; } }

        public coordinate(int x, int y)
        {
            this.x = x;
            this.y = y;
        }
    }
}
```

Coordinate class represent the object position {snake,food,grid position}

-Private int x and y → store the position

-Public int X{ get{return x;}},public int Y{get{return y;}} → this gives the read access to the outside world

-public coordinate(int x, int y)

```
{
    this.x = x;
```

```
        this.y = y;
    }
```

This is constructor , when we create object we can able to assign values through this constructor

This.x → represent class field and x represent constructor parameter

- private int x; private int y; → here i used private access modifier because for maintain Encapsulation , if they are public then this code modified directly which break the game logic

Then go to the program .cs

using Snake;

coordinate gridDimensions = new coordinate(50, 50);

coordinate snakePosition = new coordinate(25, 25);

Then

```
for (int y = 0; y < gridDimensions.Y; y++)
{
    for (int x=0; x< gridDimensions.X; x++)
    {

    }

}
```

Basically this code block visits the game board every cell

Inside the for loop add

```
if (x == 0 || y == 0 || x == gridDimensions.X - 1 || y == gridDimensions.Y - 1)
    Console.Write("=");
```

This code is print the wall or boundary

```
for (int y = 0; y < gridDimensions.Y; y++)
{
    for (int x=0; x< gridDimensions.X; x++)
    {
        if (x == 0 || y == 0 || x == gridDimensions.X - 1 || y ==
gridDimensions.Y - 1)
            Console.Write("=");
        else
            Console.Write(" ");
    }
    Console.WriteLine();
}
```

Then write the else case that if it is not in the boundary then print " "(blank) then console.write line move to the next row

In the [program.cs](#)

```
using Snake;

coordinate gridDimensions = new coordinate(100, 25);
coordinate snakePosition = new coordinate(25, 12);

for (int y = 0; y < gridDimensions.Y; y++)
{
```

```

for (int x=0; x< gridDimensions.X; x++)
{
    coordinate currentCoordinate = new coordinate(x, y);

    if (snakePosition == currentCoordinate)
        Console.Write("■");

    else if (x == 0 || y == 0 || x == gridDimensions.X - 1 || y ==
gridDimensions.Y - 1)
    {

        Console.ForegroundColor = ConsoleColor.DarkRed;
        Console.Write("=");
        Console.ResetColor();
    }
    else
    {
        Console.ForegroundColor = ConsoleColor.DarkGray;
        Console.Write(" ");
    }
}
Console.WriteLine();
}

```

Here we adds only if (snakePosition == currentCoordinate)
 Console.Write("■");
 This part

After running the code you will noticed that the snake do not appear in the console window this is because here we used == operator basically this operator compare the references not the attributes here the two objects are in the different memory

Solution We should compare the values not memory

Go to the [coordinate.cs](#) class add this

```
public override bool Equals(object? obj)
{
    if ((obj == null) || !GetType().Equals(obj.GetType()))
        return false;

    coordinate other = (coordinate)obj;
    return x == other.x && y == other.y;
}
```

Go to [program.cs](#)

Change

```
if (snakePosition == currentCoordinate)
    Console.Write("■"); To the code below
```

```
using Snake;

coordinate gridDimensions = new coordinate(100, 25);
coordinate snakePosition = new coordinate(25, 12);

for (int y = 0; y < gridDimensions.Y; y++)
{
    for (int x=0; x< gridDimensions.X; x++)
    {

        coordinate currentCoordinate = new coordinate(x, y);

        Console.ForegroundColor = ConsoleColor.DarkYellow;
        if (snakePosition.Equals( currentCoordinate))

            Console.Write("■");
    }
}
```

```

        else if (x == 0 || y == 0 || x == gridDimensions.X - 1 || y ==
gridDimensions.Y - 1)
        {

            Console.ForegroundColor = ConsoleColor.DarkRed;
            Console.Write("=");
            Console.ResetColor();
        }
        else
        {
            Console.ForegroundColor = ConsoleColor.DarkGray;
            Console.Write(" ");
        }
    }
    Console.WriteLine();
}

```

Now we have to generate randomly food position of the snake so go to the [program.cs](#)
In the top adds this

```

Random random = new Random();

coordinate gridDimensions = new coordinate(100, 25);
coordinate snakePosition = new coordinate(25, 12);

coordinate appleposition = new coordinate(random.Next(1, gridDimensions.X -
1), random.Next(1, gridDimensions.Y - 1));

```

In the [program.cs](#)
Add a frame delay and while loop as per the below in the code

```

using Snake;
using System.Drawing;

```

```
Console.OutputEncoding = System.Text.Encoding.UTF8;// this is used for the use of •
```

```
Random random = new Random();
```

```
coordinate gridDimensions = new coordinate(100, 25);
```

```
coordinate snakePosition = new coordinate(25, 12);
```

```
coordinate appleposition = new coordinate(random.Next(1, gridDimensions.X - 1), random.Next(1, gridDimensions.Y - 1));
```

```
int frameDelayMilli = 100;
```

```
while(true)
```

```
{
```

```
    Console.Clear();
```

```
for (int y = 0; y < gridDimensions.Y; y++)
```

```
{
```

```
    for (int x = 0; x < gridDimensions.X; x++)
```

```
    {
```

```
        coordinate currentCoordinate = new coordinate(x, y);
```

```
        if (snakePosition.Equals(currentCoordinate))
```

```
        {
```

```
            Console.ForegroundColor = ConsoleColor.DarkYellow;
```

```
            Console.Write("■");
```

```
        }
```

```
        else if (appleposition.Equals(currentCoordinate))
```

```
        {
```

```
            Console.ForegroundColor = ConsoleColor.Green;
```

```
            Console.Write("●");
```

```
        }
```

```
        else if (x == 0 || y == 0 || x == gridDimensions.X - 1 || y == gridDimensions.Y - 1)
```

```
        {
```

```

        Console.ForegroundColor = ConsoleColor.DarkRed;
        Console.Write("=");
        Console.ResetColor();
    }
    else
    {
        Console.ForegroundColor = ConsoleColor.DarkGray;
        Console.Write(" ");
    }
}
Console.WriteLine();
}

Thread.Sleep(frameDelayMilli);
}

```

Thread.Sleep(frameDelayMilli); → control the game speed, create animation effect, prevent cpu overload

console.clear(); this is used for the clearing the console if this is not written then the frames shows one by one down

Then we have to add the logic for the movement of the snake

We have to create a new class name [Direction.cs](#)

Add this code

```

namespace Snake
{
    internal enum Direction
    {
        Up,
        Down,
        Left,
        Right
    }
}

```


Now go to the [program.cs](#)

At the top of this add

Direction movementDirection = Direction.Down;

Then go to [coordinate.cs](#)

Add this code

```
public void ApplyMovementDirection(Direction direction)
{
    switch (direction)
    {
        case Direction.Left:
            x--;
            break;
        case Direction.Right:
            x++;
            break;
        case Direction.Up:
            y--;
            break;
        case Direction.Down:
            y++;
            break;
    }
}
```

Now when we run the game then we see that the snake moves downward but the when we pressed any key nothing happens

This is because in the above we used in the [program.cs](#)

Thread.Sleep(frameDelayMilli);

Due to this when we pressed any key or arrow in that time the program sleeps due to this reason there is no snake movement

Solution for this problem is that we have to used manually Wait in that time we have to check the keyboard weather any key pressed or not

In the [program.cs](#)

Add this code in place of Thread.Sleep(frameDelayMilli);

```

DateTime time = DateTime.Now;

while ((DateTime.Now - time).Milliseconds < frameDelayMilli)
{
    if (Console.KeyAvailable)
    {
        ConsoleKey key = Console.ReadKey().Key;

        switch (key)
        {
            case ConsoleKey.LeftArrow:
                movementDirection = Direction.Left;
                break;
            case ConsoleKey.RightArrow:
                movementDirection = Direction.Right;
                break;
            case ConsoleKey.UpArrow:
                movementDirection = Direction.Up;
                break;
            case ConsoleKey.DownArrow:
                movementDirection = Direction.Down;
                break;
        }
    }
}

```

When we run the game we can able to move the snake in the different direction
 Now the next part is to add the tail of the snake for this go to [program.cs](#)

At the top create a list that store the coordinate (previous) of the snake and make the initial length of the tail 1 so that it is one block length

```

List<coordinate> snakeposhistory = new List<coordinate>();
int tailLength = 1;

```

```

snakeposhistory.Add(new coordinate(snakePosition.X,snakePosition.Y));

```

Here we used new coordinate because , in every frame a new object is created
 If we write

```

if (snakeposhistory.Count > tailLength)
{
    snakeposhistory.RemoveAt(0);
}

```

After that add this code this code is used for the remove the element that present in the last if we do not add this code then the snake donot left his tail in the console and there is big line shows while movement

Rendering the tail on the screen

```

        coordinate currentCoordinate = new coordinate(x, y);

        //Print the tail of the snake
        if (snakePosition.Equals(currentCoordinate) ||
(snakeposhistory.Contains(currentCoordinate)))
        {
            Console.ForegroundColor = ConsoleColor.DarkYellow;
            Console.Write("■");
        }

```

After adding this code we can see that there is a small tail of the snake

Then we have to add the logic when snake eats the food or the snake position is equal to the food position then the length of the tail increases and the position of the apple shifted to a new position

```

snakeposhistory.Add(new coordinate(snakePosition.X,snakePosition.Y));
if (snakeposhistory.Count > tailLength)
{
    snakeposhistory.RemoveAt(0);
}

if (snakePosition.Equals(appleposition))
{
    tailLength= tailLength+1;
    appleposition = new coordinate(random.Next(1, gridDimensions.X -
1), random.Next(1, gridDimensions.Y - 1));
}

```

```
}
```

After that add the score logic part and collision part

In the top write **int score =0;**

Inside while loop bottom of the Console.Clear();
Write Console.WriteLine("Score" + score);

Then when the snake eat food then the score increases so

```
if (snakePosition.Equals(appleposition))
{
    tailLength= tailLength+1;
    appleposition = new coordinate(random.Next(1, gridDimensions.X -
1), random.Next(1, gridDimensions.Y - 1));
    score++;
}
```

Then we have to see the game over condition when the snake collide on the wall or when the snake itself collide then the score reset and the length of the snake reset

```
if (snakePosition.Equals(appleposition))
{
    tailLength= tailLength+1;
    appleposition = new coordinate(random.Next(1, gridDimensions.X -
1), random.Next(1, gridDimensions.Y - 1));
    score++;
}
else if (snakePosition.X == 0 || snakePosition.Y == 0 ||
snakePosition.X == gridDimensions.X - 1 || snakePosition.Y ==
gridDimensions.Y - 1 || snakeposhistory.Contains(snakePosition))
```

```

{
    score = 0;
    tailLength = 1;
    snakePosition = new coordinate(25, 12);
    snakeposhistory.Clear();
    movementDirection = Direction.Down;
    continue;
}

```

When we run the game then there is some error happen because there is a flow error the correct flow is

1.Move the snake→2. Check the collision →3. Add history

```

if (snakePosition.Equals(appleposition))
{
    tailLength= tailLength+1;
    appleposition = new coordinate(random.Next(1, gridDimensions.X - 1),
    random.Next(1, gridDimensions.Y - 1));
    score++;
}
else if (snakePosition.X == 0 || snakePosition.Y == 0 || snakePosition.X
== gridDimensions.X - 1 || snakePosition.Y == gridDimensions.Y - 1 ||
snakeposhistory.Contains(snakePosition))
{
    score = 0;
    tailLength = 1;
    snakePosition = new coordinate(25, 12);
    snakeposhistory.Clear();
    movementDirection = Direction.Down;
    continue;
}

snakeposhistory.Add(new coordinate(snakePosition.X, snakePosition.Y));
if (snakeposhistory.Count > tailLength)
{
    snakeposhistory.RemoveAt(0);
}

```

[Program.cs](#) code

```
using Snake;
using System.Drawing;
Console.OutputEncoding = System.Text.Encoding.UTF8; // this is used for the
use of •

Random random = new Random();

coordinate gridDimensions = new coordinate(60, 25);
coordinate snakePosition = new coordinate(25, 12);

Direction movementDirection = Direction.Down;

coordinate appleposition = new coordinate(random.Next(1, gridDimensions.X -
1), random.Next(1, gridDimensions.Y - 1));

int frameDelayMilli = 100;

List<coordinate> snakeposhistory = new List<coordinate>();
int tailLength = 1;

int score = 0;

while (true)
{
    Console.Clear();

    Console.WriteLine("Score" + score);

    snakePosition.ApplyMovementDirection(movementDirection);
    for (int y = 0; y < gridDimensions.Y; y++)
    {
```

```

    for (int x = 0; x < gridDimensions.X; x++)
    {

        coordinate currentCoordinate = new coordinate(x, y);

        //Print the tail of the snake
        if (snakePosition.Equals(currentCoordinate) ||
(snakeposhistory.Contains(currentCoordinate)))
        {
            Console.ForegroundColor = ConsoleColor.DarkYellow;
            Console.Write("■");
        }

        else if (appleposition.Equals(currentCoordinate))
        {
            Console.ForegroundColor = ConsoleColor.Green;
            Console.Write("●");
        }

        else if (x == 0 || y == 0 || x == gridDimensions.X - 1 || y ==
gridDimensions.Y - 1)
        {
            Console.ForegroundColor = ConsoleColor.DarkRed;
            Console.Write("=");
            Console.ResetColor();
        }
        else
        {
            Console.ForegroundColor = ConsoleColor.DarkGray;
            Console.Write(" ");
        }
    }
    Console.WriteLine();

}

if (snakePosition.Equals(appleposition))
{

```

```

        tailLength=tailLength+1;
        appleposition = new coordinate(random.Next(1, gridDimensions.X -
1), random.Next(1, gridDimensions.Y - 1));
        score++;
    }
    else if (snakePosition.X == 0 || snakePosition.Y == 0 ||
snakePosition.X == gridDimensions.X - 1 || snakePosition.Y ==
gridDimensions.Y - 1 || snakeposhistory.Contains(snakePosition))
    {
        score = 0;
        tailLength = 1;
        snakePosition = new coordinate(25, 12);
        snakeposhistory.Clear();
        movementDirection = Direction.Down;
        continue;
    }

    snakeposhistory.Add(new coordinate(snakePosition.X, snakePosition.Y));
    if (snakeposhistory.Count > tailLength)
    {
        snakeposhistory.RemoveAt(0);
    }

    DateTime time = DateTime.Now;

    while ((DateTime.Now - time).TotalMilliseconds < frameDelayMilli)
    {
        if (Console.KeyAvailable)
        {
            ConsoleKey key = Console.ReadKey().Key;

            switch (key)
            {
                case ConsoleKey.LeftArrow:
                    movementDirection = Direction.Left;
                    break;
                case ConsoleKey.RightArrow:
                    movementDirection = Direction.Right;
                    break;
                case ConsoleKey.UpArrow:
                    movementDirection = Direction.Up;
                    break;
                case ConsoleKey.DownArrow:
                    movementDirection = Direction.Down;
                    break;
            }
        }
    }
}

```



```
    }  
  }  
}
```

[coordinate.cs](#) code

```
using System;  
using System.Collections.Generic;  
using System.Drawing;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
  
namespace Snake  
{  
    internal class coordinate  
    {  
  
        private int x;  
        private int y;  
  
        public int X { get { return x; } }  
        public int Y { get { return y; } }  
  
        public coordinate(int x, int y)  
        {  
            this.x = x;  
            this.y = y;  
        }  
  
        public override bool Equals(object? obj)  
        {  
            if ((obj == null) || !GetType().Equals(obj.GetType()))  
                return false;  
  
            coordinate other = (coordinate)obj;  
            return x == other.x && y == other.y;  
        }  
  
        public void ApplyMovementDirection(Direction direction)  
        {
```

```
        switch (direction)
        {
            case Direction.Left:
                x--;
                break;
            case Direction.Right:
                x++;
                break;
            case Direction.Up:
                y--;
                break;
            case Direction.Down:
                y++;
                break;
        }
    }
}
```

[Direction.cs](#) code

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Snake
{
    internal enum Direction
    {
        Up,
        Down,
        Left,
        Right
    }
}
```

When we starts it shows like this



This is very basic

Now we have to add the the intro of the game and the end when the snake collide

In the [program.cs](#)

Write at the top

```
bool gameover = false;
```

Then above while Loop

Write this is the intro of the game

```
static void ShowIntro()
{
    Console.Clear();
    Console.WriteLine("==== SNAKE GAME =====");
    Console.WriteLine();
    Console.WriteLine("Press P to Play");
}
```

```

    Console.WriteLine("Press E to Exit");

    ConsoleKey key = Console.ReadKey(true).Key;

    if (key == ConsoleKey.E)
        Environment.Exit(0);
}

```

Then

```
ShowIntro();
```

This is loading

```

static void ShowLoading()
{
    Console.Clear();
    Console.Write("Loading");

    for (int i = 0; i < 5; i++)
    {
        Thread.Sleep(400);
        Console.Write(".");
    }

    Thread.Sleep(700);
}

ShowLoading();

```

Then we have to fixed the game over logic

Change this

```

    else if (snakePosition.X == 0 || snakePosition.Y == 0 ||
snakePosition.X == gridDimensions.X - 1 || snakePosition.Y ==
gridDimensions.Y - 1 || snakeposhistory.Contains(snakePosition))
    {
        //score = 0;
        //tailLength = 1;
        //snakePosition = new coordinate(25, 12);
    }

```

```

        //snakeposhistory.Clear();
        //movementDirection = Direction.Down;
        //continue;

        gameover = true;
    }

```

```

snakeposhistory.Add(new coordinate(snakePosition.X, snakePosition.Y));
if (snakeposhistory.Count > tailLength)
{
    snakeposhistory.RemoveAt(0);
}

////////////////////////////////////// this is
the logic
if (gameover)
{
    Console.Clear();
    Console.WriteLine("==== GAME OVER =====");
    Console.WriteLine("Final Score: " + score);
    Console.WriteLine();
    Console.WriteLine("Press R to Restart");
    Console.WriteLine("Press E to Exit");

    ConsoleKey key = Console.ReadKey(true).Key;

    if (key == ConsoleKey.E)
        Environment.Exit(0);

    // Reset game values
    score = 0;
    tailLength = 1;
    snakePosition = new coordinate(25, 12);
    snakeposhistory.Clear();
    movementDirection = Direction.Down;
    gameover = false;

    continue;
}

```

```
}
```

```
////////////////////////////////////
```

```
===== SNAKE GAME =====
```

```
Press P to Play
```

```
Press E to Exit
```



This is the final code of [program.cs](#) in this there some changes for the better visulation.

```
using Snake;
using System.Drawing;
Console.OutputEncoding = System.Text.Encoding.UTF8; // this is used for the
use of •

bool gameover=false;

Random random = new Random();

coordinate gridDimensions = new coordinate(60, 25);
coordinate snakePosition = new coordinate(25, 12);

Direction movementDirection = Direction.Down;

coordinate appleposition = new coordinate(random.Next(1, gridDimensions.X -
1), random.Next(1, gridDimensions.Y - 1));
```

```
int frameDelayMilli = 100;

List<coordinate> snakeposhistory = new List<coordinate>();
int tailLength = 1;

int score = 0;

static void ShowIntro()
{
    Console.Clear();
    Console.WriteLine("==== SNAKE GAME =====");
    Console.WriteLine();
    Console.WriteLine("Press P to Play");
    Console.WriteLine("Press E to Exit");

    ConsoleKey key = Console.ReadKey(true).Key;

    if (key == ConsoleKey.E)
        Environment.Exit(0);
}

ShowIntro();

static void ShowLoading()
{
    Console.Clear();
    Console.Write("Loading");

    for (int i = 0; i < 5; i++)
    {
        Thread.Sleep(400);
        Console.Write(".");
    }

    Thread.Sleep(700);
}

ShowLoading();
```

```

while (true)
{
    Console.Clear();

    Console.WriteLine("Score" + score);

    snakePosition.ApplyMovementDirection(movementDirection);
    for (int y = 0; y < gridDimensions.Y; y++)
    {
        for (int x = 0; x < gridDimensions.X; x++)
        {

            coordinate currentCoordinate = new coordinate(x, y);

            //Print the tail of the snake
            if (snakePosition.Equals(currentCoordinate) ||
(snakeposhistory.Contains(currentCoordinate)))
            {
                Console.ForegroundColor = ConsoleColor.DarkYellow;
                Console.Write("■");
            }

            else if (appleposition.Equals(currentCoordinate))
            {
                Console.ForegroundColor = ConsoleColor.Green;
                Console.Write("●");
            }

            else if (x == 0 || y == 0 || x == gridDimensions.X - 1 || y ==
gridDimensions.Y - 1)
            {

                Console.ForegroundColor = ConsoleColor.DarkRed;
                Console.Write("■");
                Console.ResetColor();
            }
            else
            {
                Console.ForegroundColor = ConsoleColor.DarkGray;

```



```

        Console.Write(" ");
    }
}
Console.WriteLine();

}

if (snakePosition.Equals(appleposition))
{
    tailLength= tailLength+1;
    appleposition = new coordinate(random.Next(1, gridDimensions.X -
1), random.Next(1, gridDimensions.Y - 1));
    score++;
}
else if (snakePosition.X == 0 || snakePosition.Y == 0 ||
snakePosition.X == gridDimensions.X - 1 || snakePosition.Y ==
gridDimensions.Y - 1 || snakeposhistory.Contains(snakePosition))
{
    //score = 0;
    //tailLength = 1;
    //snakePosition = new coordinate(25, 12);
    //snakeposhistory.Clear();
    //movementDirection = Direction.Down;
    //continue;

    gameover = true;
}

snakeposhistory.Add(new coordinate(snakePosition.X, snakePosition.Y));
if (snakeposhistory.Count > tailLength)
{
    snakeposhistory.RemoveAt(0);
}

////////////////////////////////////
if (gameover)
{
    Console.Clear();
    Console.WriteLine("==== GAME OVER =====");
    Console.WriteLine("Final Score: " + score);
}

```

```

    Console.WriteLine();
    Console.WriteLine("Press R to Restart");
    Console.WriteLine("Press E to Exit");

    ConsoleKey key = Console.ReadKey(true).Key;

    if (key == ConsoleKey.E)
        Environment.Exit(0);

    // Reset game values
    score = 0;
    tailLength = 1;
    snakePosition = new coordinate(25, 12);
    snakeposhistory.Clear();
    movementDirection = Direction.Down;
    gameover = false;

    continue;
}

////////////////////////////////////

DateTime time = DateTime.Now;

while ((DateTime.Now - time).TotalMilliseconds < frameDelayMilli)
{
    if (Console.KeyAvailable)
    {
        ConsoleKey key = Console.ReadKey().Key;

        switch (key)
        {
            case ConsoleKey.LeftArrow:
                movementDirection = Direction.Left;
                break;
            case ConsoleKey.RightArrow:
                movementDirection = Direction.Right;
                break;
            case ConsoleKey.UpArrow:
                movementDirection = Direction.Up;
                break;
            case ConsoleKey.DownArrow:
                movementDirection = Direction.Down;
                break;
        }
    }
}

```

```
}  
  }  
}
```